88.6
Pass

Harshal Kolhe

PDF generated at: 24 Feb 2026 06:55:37 UTC

View this report on HackerRank

Score

88.6% • 155.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 64 min 28 sec on 24 Feb 2026 11:18:58 IST

Candidate Information

Email	harshalkolhe0@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	24 Feb 2026 11:18:58 IST
Time taken	64 min 28 sec/ 90 min
Work Experience	5 years
Invited by	Vipul
Invited on	23 Feb 2026 16:19:32 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	0%

2	Spring Boot Intermediate	33%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	20%	Hard	0%
Spring	0%	Application Properties Annotations	0%
Medium	93%	Exception Handling	100%
Aspect-Oriented Programming	100%	Sorting	100%
Greedy	100%	Arrays	100%
Two Pointers	100%	Java	100%

Questions

Spring Boot (Advanced) • 5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	JSON Deserialization Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min 28 sec	⊗

2	Specification Multiple Choice	Spring Boot (Advanced)	0/5	-	40 sec	✗
3	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	0/5	-	2 min 11 sec	✗
4	CSRF Multiple Choice	Spring Boot (Intermediate)	0/5	-	51 sec	✗
5	Predict the Output Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 33 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Capable Models Coding	Problem Solving (Intermediate)	75/75	-	26 min 46 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Optimized Sequential Video Watching Coding	Java (Intermediate)	75/75	-	29 min 17 sec	✓

1. JSON Deserialization

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

CheckerCreateRequest.java

```
// Annotation here
public class CheckerCreateRequest {
    private String name;
    private String surname;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getSurname() {
        return surname;
    }
    public void setSurname(String surname) {
        this.surname = surname;
    }
}
```

CheckerController.java

```
@RestController
@RequestMapping("/checkers")
public class CheckerController {

    @Autowired
    CheckerService checkerService;

    @PostMapping
    public ResponseEntity createChecker(@RequestBody CheckerCreateRequest checkerCreateReque
st) {
        checkerService.createChecker(checkerCreateRequest);
        return new ResponseEntity(HttpStatus.CREATED);
    }
}
```

Request Json data

```
{
   "checker":{
      "name": "Checker",
      "surname":"Number2"
   }
}
```

A POST request with "Request Json data" is made to "/checkers". What annotation needs to be added to the "CheckerCreateRequest" class so that it can successfully deserialize and process the request?

Interviewer guidelines

@JsonRootName doesn't add checker as a wrapper attribute by itself. To make it successfully deserialize with given json annotations on D should be used together.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ @JsonRootName("checker")
- ○ @JsonName("checker")
- ● **@JsonTypeName("checker")**
- ○ @JsonTypeInfo(include = JsonTypeInfo.As.WRAPPER_OBJECT, use = JsonTypeInfo.Id.NAME) @JsonTypeName("checker") ✓

No comments.

2. Specification

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- PreparedStatement
- Specification
- Criteria
- Query

No comments.

3. Configuration Properties

Incorrect

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code

Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] ** The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2). **
- [ ] The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).
- [ ] A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final.
- [ ] An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.
- [ ] A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

4. CSRF

Incorrect

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ private void manage(HttpSecurity http) {   http.csrf().allowed() }
- ○ private void manage(HttpSecurity http) {   http.csrf() }
- ● ** private void manage(HttpSecurity http) {   http.csrf().enable() }   **
- ○ None of the above. CSRF is configured by default when Spring Security Dependency is added. ✓

No comments.

5. Predict the Output

✓ Correct

Multiple Choice | Medium | Exception Handling | Spring Boot | Aspect-Oriented Programming

Question description

Consider the following Snippet and predict the output.

The following API is triggered: http://localhost:8080/add/employee?eId=123&fName=Ravi&lastName=Shekhar

```
@Autowired
private EService eService;

@RequestMapping(value = "/add/employee", method = RequestMethod.GET)
public com.model.Employee add(@RequestParam("eId") String eId, @RequestParam("fName") Strin
g fName, @RequestParam("lastName") String lastName)
{
   return eService.createEmployee(eId, fName, lastName);
}

@Aspect
@Component
public class EServiceAspect
{
   @After(value = "execution(* com.random.service.EService.*(..)) && args(eId, fName, lastName)")
   public void afterAdvice(JoinPoint joinPoint, String eId, String fName, String lastName) {
      System.out.println("After method:" + joinPoint.getSignature());
      System.out.println("Creating Employee with first name - " + fName+ ", second name - " + lastNa
me+ " and id - " + eId);
   }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(string,string,string) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(empId,firstName,secondName) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Shekhar, second name - Ravi and id - 123

No comments.

6. Capable Models

Correct

Coding | Medium | Sorting | Greedy | Arrays | Two Pointers

Question description

Given *n* machine learning models, each with an associated cost and feature compatibility:

- *cost[i]* represents the cost of the ith model

- *featureAvailability[i]* is a binary string indicating suitability for two distinct features:
  - "00": not equipped for either feature
  - "01": suitable for feature A but not feature B
  - "10": suitable for feature B but not feature A
  - "11": suitable for both features

A set of models is *k*-capable if the number of models suitable for feature A and the number suitable for feature B are both greater than or equal to *k*.

For each value of *k* from 1 to *n*, determine the minimum cost required to assemble a *k*-capable set of models. Return an array of *n* integers, where the ith integer represents the minimum cost for an *i*-capable set. If no *i*-capable set exists, the ith integer should be -1.

Example

n = 6

cost = [3, 6, 9, 1, 2, 5]

featureAvailability = ["10", "01", "11", "01", "11", "10"]

featureAvailability = [“10”, “01”, “11”, “01”, “11”, “10”]

k	Optimal Set	Feature 1 Compatible	Feature 2 Compatible	Cost
1	[5]	[5]	[5]	2
2	[1, 4, 5]	[1, 5]	[4, 5]	3 + 1 + 2 = 6
3	[1, 3, 4, 5]	[1, 3, 5]	[3, 4, 5]	3 + 9 + 1 + 2 = 15
4	[1, 2, 3, 4, 5, 6]	[1, 3, 5, 6]	[2, 3, 4, 5]	3 + 6 + 9 + 1 + 2 + 5 = 26

For $k \geq 5$, there will be no capable set.

Hence, the answer is [2, 6, 15, 26, -1, -1].

Function Description

Complete the function *getMinimumCost* in the editor with the following parameters:

int cost[n]: the cost of machine learning models

string featureAvailability[n]: the compatibility string of models indicating their suitability for two features

Returns

int[n]: the ith integer is the minimum cost of a set of *i*-capable models

Constraints

- $1 \le n \le 10^5$
- $1 \le cost[i] \le 10^4$
- It is guaranteed that *featureAvailability[i]* is a binary string of length 2.

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of machine learning models.
Each of the next *n* lines contains an integer *cost[i]*.
The next line contains an integer, *n*, the number of models.
Each of the next *n* lines contains a string *featureAvailability[i]*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN        FUNCTION
-----        --------
4      →   cost[] size n = 4
5      →   cost = [5, 6, 10, 1]
6
10
1
4      →   featureAvailability[] size n = 4
10      →   featureAvailability = ["10", "01", "11", "00"]
01
11
00
```

Sample Output

```
10
21
-1
-1
```

Explanation

k	Optimal Set	Feature 1 Compatible	Feature 2 Compatible	Cost

1	[3]	[3]	[3]	10
2	[1, 2, 3]	[1, 3]	[2, 3]	5 + 6 + 10 = 21

For $k \geq 3$, there is no capable set.

Hence, the answer is [10, 21, -1, -1].

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN        FUNCTION
-----        --------
2       →    cost[] size n = 2
1       →    cost = [1, 1]
1
2       →    featureAvailability[] size n = 2
10      →    featureAvailability = ["10", "10"]
10
```

Sample Output

```
-1
-1
```

Explanation

There is no possible way of forming a *k-capable* set for all *k* from 1 to *n*.

Interviewer guidelines

▼ SOLUTION

Skills: Sorting, Two Pointers, Greedy Algorithms

To find the minimum cost for models such that the system is k-capable, the ones with "00" are unusable, while "01" and "10" have to be used together always and "11" can be used independently. This is because there should be k models supporting both features A and B.

Also, to find for each k from 1 to n, if the minimum cost of a k-capable system is known, find the next unused model(s) with minimum cost such that it supports both features A and B, and add the cost to find the minimum cost of (k + 1)-capable system.

So, the target is to create an array in increasing order such that each element is the cost of adding model(s) such that it supports both features.

Create three arrays: cost_both containing the models “11”, cost_one containing the models “01”, and cost_two containing the models “10”. Then, sort these arrays in an increasing order. Then, create an array cost_sum such that the ith element in cost_sum is cost_one[i] + cost_two[i], representing the cost to convert one “01” and one “10” into one “11” in sorted order. Then, append the two arrays cost_both and cost_sum and sort them to get the final array of costs.

Optimal Solution:

```
def getMinimumCost(cost, featureAvailability):
    n = len(cost)

    x1 = [x for x, y in zip(cost, featureAvailability) if y == "10"]
    x2 = [x for x, y in zip(cost, featureAvailability) if y == "01"]

    x = sorted(map(sum, zip(sorted(x1), sorted(x2))), reverse=True)
    y = sorted([x for x, y in zip(cost, featureAvailability) if y == "11"], reverse=True)

    ans = []
    px = py = val = 0
    while x or y:
        if not x or (x and y and x[-1] > y[-1]):
            val += y.pop()
        else:
            val += x.pop()
        ans.append(val)
    return ans + [-1] * (n - len(ans))
```

▼ COMPLEXITY ANALYSIS

Time Complexity - O(n log n)

The sorting algorithm takes O(n log n) which is the highest order of operations performed. Therefore, the time complexity is O(n log n).

Space Complexity - O(n)

There are different auxiliary arrays used to store the costs. Therefore, the space complexity is O(n).

Candidate's Solution

Language used: **Java 17**

```
import java.io.*;
import java.math.*;
```

```java
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getMinimumCost' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY cost
     *  2. STRING_ARRAY featureAvailability
     */

    public static List<Integer> getMinimumCost(List<Integer> cost, List<String> featureAvailability) {
    // Write your code here
        List<int[]> first=new ArrayList<>();
        List<int[]> second=new ArrayList<>();
        List<int[]> both=new ArrayList<>();
        List<Integer> result=new ArrayList<>();
        for(int i=0;i<featureAvailability.size();i++){
            if(featureAvailability.get(i).charAt(0)=='1'&&
                featureAvailability.get(i).charAt(1)=='1')   both.add(new int[]{cost.get(i),i});
            else if(featureAvailability.get(i).charAt(0)=='1')   {
                //System.out.println("i="+i+", "+featureAvailability.get(i));
                first.add(new int[]{cost.get(i),i});
            }
            else if(featureAvailability.get(i).charAt(1)=='1') second.add(new int[]{cost.get(i),i});
        }
        first.sort((a,b)->a[0]-b[0]);
        second.sort((a,b)->a[0]-b[0]);
        int i=0,j=0,n=first.size(),m=second.size();
        PriorityQueue<Integer> minQueue=new PriorityQueue<>();
        while(i<n&&j<m){
```

```
            minQueue.add(first.get(i)[0]+second.get(j)[0]);
            i++;j++;
        }
        for(int[] b:both){
            minQueue.add(b[0]);
        }
        int sum=0;
        while(!minQueue.isEmpty()){
            sum+=minQueue.poll();
            result.add(sum);
        }
        while(result.size()<cost.size())result.add(-1);
        return result;
    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int costCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> cost = IntStream.range(0, costCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int featureAvailabilityCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<String> featureAvailability = IntStream.range(0, featureAvailabilityCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine();
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
```

```
        })
            .collect(toList());

        List<Integer> result = Result.getMinimumCost(cost, featureAvailability);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0581 sec	33.6 KB
TestCase 1	Easy	Sample	Success	1	0.051 sec	34 KB
TestCase 2	Easy	Sample	Success	1	0.0503 sec	33.8 KB
TestCase 3	Easy	Hidden	Success	2	0.06 sec	34 KB
TestCase 4	Easy	Hidden	Success	2	0.0902 sec	34.1 KB
TestCase 5	Medium	Hidden	Success	5	0.2354 sec	49.7 KB
TestCase 6	Medium	Hidden	Success	5	0.2151 sec	53.3 KB

TestCase 7	Medium	Hidden	Success	5	0.1915 sec	48.8 KB
TestCase 8	Medium	Hidden	Success	5	0.2228 sec	52.3 KB
TestCase 9	Hard	Hidden	Success	8	0.2432 sec	57 KB
TestCase 10	Hard	Hidden	Success	8	0.4709 sec	64.4 KB
TestCase 11	Hard	Hidden	Success	8	0.619 sec	75.6 KB
TestCase 12	Hard	Hidden	Success	8	0.5122 sec	68.6 KB
TestCase 13	Hard	Hidden	Success	8	0.6701 sec	76.5 KB
TestCase 14	Hard	Hidden	Success	8	0.5631 sec	81.8 KB

No comments.

7. Java: Optimized Sequential Video Watching

Correct

Coding Medium Greedy Java

Question description

A learner is given a sequential playlist of *n* videos to enhance their knowledge. Implement a function to calculate the minimum time required to complete a total of *m* viewings while adhering to the platform's

constraints.

System Constraints

- Sequential Viewing:
  - The learner must watch videos in the order they appear in the playlist.
  - To watch a video at index *i*, the learner must first watch all videos from index 0 to *i-1* at least once.
- Initial Full Viewing:
  - To view any video at index *i* for the first time, the learner must spend *firstWatch[i] + repeatWatch[i]* minutes. This ensures a complete understanding of the video.
- Subsequent Viewings:
  - For subsequent viewings of a video at index *i*, the learner spends only *repeatWatch[i]* minutes.
  - At any time, they can rewatch any video they have already watched.
- Minimum Total Watch Count:
  - The learner must achieve at least *m* total viewings across all videos.
  - Videos can be rewatched as desired, and not all videos need to be watched equally.

Return the minimum time required to complete *m* viewings of videos while following these constraints.

Example

If *n* = 4, *m* = 4, *firstWatch* = [1, 5, 9, 11], and *repeatWatch* = [2, 7, 10, 11], the optimal strategy is as follows:

- The learner watches the 0th video for the first time, spending *firstWatch*[0] + *repeatWatch*[0] = 1 + 2 = 3 minutes.
- The learner then rewatches the 0th video 3 more times in, spending *repeatWatch*[0]*3 = 2 * 3 = 6 minutes.
- The learner has successfully completed 4 viewings. Thus, the total time required is 3 + 6 = 9 minutes, and 9 is returned.

Function Description

Complete the function *findMinMinutes* in the editor below.

Function Parameters

int m: the total number of video viewings to complete

int firstWatch[n]: the minutes required to watch each video the first time

int repeatWatch[n]: the minutes required to rewatch each video

Returns

long: the minimum minutes required for the learner to reach their target number of viewings

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 10^9$
- $1 \le firstWatch[i], repeatWatch[i] \le 10^9$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *m*.
The next line contains an integer, *n,* the size of *firstWatch*.
Each of the next *n* lines contains an integer, *firstWatch[i]*.
The next line contains an integer, *n,* the size of *repeatWatch*.
Each of the next *n* lines contains an integer, *repeatWatch[i]*.

▼ SAMPLE CASE 0

Sample Input 0

```
STDIN    FUNCTION
-----    --------
5        m = 5
3        firstWatch[] size n = 3
6        firstWatch = [6, 8, 9]
8
9
3        repeatWatch[] size n = 3
4        repeatWatch = [4, 2, 10]
2
10
```

Sample Output 0

```
26
```

Explanation

The goal is to complete 5 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time, so the time required is *firstWatch*[0] + *repeatWatch*[0] = 6 + 4 = 10 minutes.

- The learner watches the 1st video for the first time, so the time required is *firstWatch*[1] + *repeatWatch*[1] = 8 + 2 = 10 minutes.
- The learner then rewatches the 1st video 3 times. So the time taken = *repeatWatch*[1]*3 = 2 * 3 = 6 minutes.
- The total time required to complete 5 video viewings is 10 + 10 + 6 = 26 minutes.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
6         m = 6
5         firstWatch[] size n = 5
2         firstWatch = [2, 3, 7, 8, 4]
3
7
8
4
5         repeatWatch[] size n = 5
3         repeatWatch = [3, 6, 8, 7, 6]
6
8
7
6
```

Sample Output 1

```
20
```

Explanation

The goal is to complete 6 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time in *firstWatch*[0] + *repeatWatch[0]* = 2 + 3 = 5 minutes.
- The learner then rewatches the 0th video 5 times in *repeatWatch[0]**5 = 3 * 5 = 15 minutes.
- The total time required to complete 6 viewings is 5 + 15 = 20 minutes.

Interviewer guidelines

▼ SOLUTION

Description

The greedy solution suggests that, at any given index, if the time to watch the video is less than at any other index between i+1 and n, the minimum total time will be the sum of the watch time up to index i-1, plus the repeat time of the ith video multiplied by (m-i).

Solution

```
class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
        long minMinutes = Long.MAX_VALUE;
        long cur = 0;
        int n = firstWatch.size();

        for (int i = 0; i < Math.min(n,m); i++) {
            int a = firstWatch.get(i);
            int b = repeatWatch.get(i);

            minMinutes = Math.min(minMinutes, cur + ((long) (m - i) * b) + a);

            cur += a + b;
        }
        return minMinutes;
    }

}
```

▼ COMPLEXITY

Time Complexity - O(n)

The algorithm iterates through each number in *firstWatch* and *repeatWatch* exactly once, performing constant-time checks and arithmetic operations in each iteration. Thus, the time complexity is O(n), where *n* is the length of *firstWatch*.

Space Complexity - O(1)

Space usage is constant, only utilizing a fixed number of variables (*mn, cur,* etc.) that do not scale with the length of *firstWatch*. Therefore, the space complexity is O(1).

Candidate's Solution

Language used: **Java 17**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
        long result=Long.MAX_VALUE;
        long []sum=new long[firstWatch.size()];
        sum[0]=firstWatch.get(0)+repeatWatch.get(0);
        for(int i=1;i<firstWatch.size();i++){
            sum[i]=sum[i-1]+firstWatch.get(i)+repeatWatch.get(i);
        }
        for(int i=firstWatch.size()-1;i>=0;i--){
            long current=sum[i];
            if(m>i+1){
                long extra=(m-i-1)*1L*repeatWatch.get(i);
                current+=extra;
            }
            //System.out.println(current+" "+result);
```

```
            if(result>current)
                result=current;
        }
        return result;
    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int m = Integer.parseInt(bufferedReader.readLine().trim());

        int firstWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> firstWatch = IntStream.range(0, firstWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int repeatWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> repeatWatch = IntStream.range(0, repeatWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

```

```
        long result = Result.findMinMinutes(m, firstWatch, repeatWatch);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 2	Easy	Sample	Success	1	0.0604 sec	33.5 KB
Testcase 0	Easy	Sample	Success	1	0.0557 sec	33.9 KB
Testcase 1	Easy	Sample	Success	1	0.0714 sec	33.6 KB
Testcase 3	Easy	Hidden	Success	4	0.0648 sec	33.7 KB
Testcase 4	Easy	Hidden	Success	4	0.0685 sec	34 KB
Testcase 5	Easy	Hidden	Success	4	0.1002 sec	41.8 KB
Testcase 6	Medium	Hidden	Success	5	0.352 sec	63.3 KB
Testcase 7	Medium	Hidden	Success	5	0.7115 sec	164 KB
Testcase 8	Medium	Hidden	Success	5	0.4498 sec	145 KB

Testcase 9	Medium	Hidden	Success	5	0.5605 sec	140 KB
Testcase 10	Medium	Hidden	Success	5	0.6747 sec	164 KB
Testcase 11	Medium	Hidden	Success	5	0.4375 sec	150 KB
Testcase 12	Hard	Hidden	Success	10	0.3066 sec	63.5 KB
Testcase 13	Hard	Hidden	Success	10	0.471 sec	137 KB
Testcase 14	Hard	Hidden	Success	10	0.0638 sec	33.6 KB

No comments.